Week 5: Logistic and Softmax Regression*

Instructor: Dr. Federico Fusco

October 25, 2025 - version 1

1 The Perceptron Algorithm

We introduce one of the seminal machine learning algorithms: the **Perceptron**. It performs binary classification via separating hyperplanes, under the assumption that the underlying dataset is separable, i.e., there exists a hyperplane that perfectly separates the points corresponding to the two labels.

Formally, we are given n points $(x_1, y_1), (x_2, y_2) \ldots, (x_n, y_n)$, with $x_i \in \mathcal{X} = \mathbb{R}^d$, $y_i \in \mathcal{Y} = \{0, 1\}$, and we want to find an hyperplane H_θ that separates them, i.e., the points with label 1 and those with label -1 have to lie in the two distinct hyperspaces identified by H_θ. Recall, a generic hyperplane H_θ is given by all the points orthogonal to a non-zero vector θ: $H_\theta = \{x \in \mathcal{X} | x^T\theta = 0\}$. Given a vector $\theta \in \mathbb{R}^d$ and a point $x \in \mathbb{R}^d$. The label y predicted for x by the θ is then 1 if $x^T\theta \geq 0$, and 0 if $x^T\theta < 0$ (when there is equality, we can actually decide the label arbitrarily, here we have chosen label $+1$ out of simplicity).

Note

The scalar product $x^T\theta$ is equal to the cosine of the angle between x and θ, multiplied by the norm of the two vectors. If the cosine is positive, then it means that x belongs to the same side of the hyperspace as θ; if it is negative, then it belongs to the opposite hyperspace. If the scalar product is zero, then x is orthogonal to θ, and thus belongs to the hyperplane H_θ (and we can decide arbitrarily on its label).

Definition 1.1. *Given a vector θ, and a point x with label y, we say that H_θ makes a **classification error** on x if $y \cdot x^T\theta < 0$. If we have that $y \cdot x^T\theta < 1$, then we say that there is a **margin error**.*

Note, if θ induces a hyperplane that classifies correctly a point but makes a margin error, then it is enough to rescale θ conveniently to avoid the margin error. Clearly, by rescaling θ with a positive constant, the hyperplane and its orientation (i.e., which side is associated with $+1$ and which one with -1) do not change.

Remark 1.2. *As we have already done for linear regression, we are considering homogeneous hyperplanes, i.e., those that contain the origin. This is, again, without loss of generality, as we can always add an extra dimension and set the corresponding coordinate to 1 for all the points in the*

*These notes are meant as a teaching support for the students of the "Machine Learning" course for the Master's Degree in Engineering in Computer Science and Artificial Intelligence that I am teaching in Fall 2025. They are likely full of typos and imprecisions, so please get in touch with me in class if something feels off. From time to time, I may update new versions of these notes, so make sure to download the latest version.

The Perceptron Algorithm

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$
Hyperparameter: Maximum number of iterations N
Initialize $\theta_0 \leftarrow \mathbf{0} \in \mathbb{R}^d$
for $t = 1, 2, \ldots, N$ **do**
  Let i_t be an index drawn uniformly at random in $1, \ldots, n$
  if $y_{i_t} \cdot x_{i_t}^T \theta < 1$ **then** ▷ Margin Error
    $\theta_{t+1} \leftarrow \theta_t + y_{i_t} x_{i_t}$ ▷ θ_t is adjusted according to point (x_{i_t}, y_{i_t})
  else
    $\theta_{t+1} \leftarrow \theta_t$ ▷ θ_t is not updated
return θ

dataset. For instance, if we are looking at point $(x^{(1)}, x^{(2)})$ in $\mathbb{R}^2$, we can add a third coordinate $(x^{(1)}, x^{(2)}, 1)$ and consider the affine line (in $\mathbb{R}^2$) given by $(\theta^{(1)}, \theta^{(2)}, \theta^{(3)})$ as an homogeomeous line in $\mathbb{R}^3$.

The Perceptron algorithm, described in the pseudocode, works as follows. It maintains a candidate hyperplane identified by vector θ_t, then considers the points in the dataset one after the other (either cyclically, or uniformly at random as reported here). Every time that the current solution θ_t makes a margin error on the point (x_{i_t}, y_{i_t}) considered, it modifies θ_t by adding $y_{i_t} x_{i_t}$. Before stating the bound on the number of mistakes performed by the algorithm, we introduce some notions. Given a point x and an hyperplane H_θ, the distance of x to the hyperplane is:

$$\text{dist}(x, H_\theta) = \min_{z \in H_\theta} ||x - z|| = \frac{|x^T \theta|}{||\theta||}.$$

The first equality is the definition of the distance between a point and a hyperplane, while the second equality provides a simple way of computing such a distance. We give such a formula for granted, but it follows from the well-known fact that the segment joining x to its closest point on the hyperplane is orthogonal to the hyperplane itself (and is thus aligned with θ).

Definition 1.3. *Given $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ and an hyperplane H_θ that correctly classifies all the points, the margin of H_θ, denoted with $\gamma(\theta)$ is defined as the distance of H_θ from its closest point:*

$$\gamma(\theta) = \min_{i=1,\ldots,n} \text{dist}(x_i, H_\theta).$$

The largest possible margin achievable by a hyperplane that classifies the points correctly is denoted with $\gamma^\star$.

We are ready to provide an upper bound on the number of margin mistakes that the Perceptron Algorithm makes. We say that the algorithm converges if it finds a hyperplane that correctly classifies all the points and makes no margin error. Finally, we denote with D the diameter of the problem, i.e. $\max_i ||x_i||$.

Theorem 1.4. *The Perceptron Algorithm makes at most $(2+D^2)/(\gamma^*)^2$ margin mistakes, if the points are linearly separable.*

Proof. Consider a generic time step t where the solution θ_t makes a margin error on the point considered (x_{i_t}, y_{i_t}), we have:

$$||\theta_{t+1}||^2 = ||\theta_t + y_{i_t} x_{i_t}||^2 = ||\theta_t||^2 + 2 y_{i_t} x_{i_t} \theta_t + ||x_{i_t}||^2 \leq ||\theta_t||^2 + 2 + D^2,$$

where the inequality follows by the fact that $||x|| \leq D$ by definition of the diameter D, and by the fact that θ_t makes a margin mistake on point i_t, and thus $y_{i_t} x_{i_t}^T \theta_t$ is at most 1. Denote with m_t the number of margin error made up to time t included, if we repeatedly apply the above inequality to all the m_t errors, we get:

$$||\theta_{t+1}||^2 \leq m_t(2 + D^2) + ||\theta_0|| = m_t(2 + D^2). \tag{1}$$

Denote now with $\theta^\star$ the unit norm vector that identifies the hyperplane with the largest margin, so that $\gamma^\star = \gamma(\theta^\star)$, we have:

$$\begin{aligned}
(\theta^\star)^T(\theta_{t+1} - \theta_t) &= (\theta^\star)^T(y_{i_t} x_{i_t}) && \text{(By design of the Perceptron algorithm)} \\
&= |(\theta^\star)^T x_{i_t}| && \text{(By definition } H_{\theta^\star} \text{ correctly classifies all the points)} \\
&= \frac{|(\theta^\star)^T x_{i_t}|}{||\theta^\star||} && (\theta^\star \text{ has unit norm}) \\
&\geq \gamma^\star. && \text{(By definition of margin)}
\end{aligned}$$

This formula is useful to lower bound the norm of the generic θ_t where there is a margin error:

$$||\theta_{t+1}|| \geq (\theta^\star)^T \theta_{t+1} = \sum_{s=1}^{t} (\theta^\star)^T(\theta_{s+1} - \theta_s) \geq m_t \gamma^\star. \tag{2}$$

Note, the first inequality follows from the fact that $a^T b = \cos\alpha \cdot ||a|| \cdot ||b|| \leq ||a|| \cdot ||b||$, where α is the angle between the two vectors, and its cosine is at most 1. Combining Equation (2) with Equation (1), we get that the number of mistakes is never more than the quantity in the statement. This concludes the proof. □

Note, we have provided an upper bound on the number of errors, which is not directly an upper bound on the worst-case running time of the Perceptron algorithm. Such running time depends crucially on how the points are considered. If they are considered uniformly at random one after the other, then we can say that the expected running time is $O(nM)$, where M is the number of mistakes (if we stop when all the points are correctly classified or there are no margin errors). If instead we cycle through the points and stop when all the points are correctly classified (or there is no margin error), then the worst-case running time is $O(nM)$.

2 Support Vector Machine

For the perceptron algorithm to work, we need to assume separability, namely, that there exists a θ such that $x_i^T \theta y_i > 0$ for every point in the dataset. To normalize the vectors θ, we have also introduced a notion of margin error when $x_i^T \theta y_i < 1$, and demanded that the output θ respects it. Another way of looking at this classification task is to solve the following *hard-margin* problem:

$$\begin{aligned}
\min_{\theta} \quad & \frac{1}{2}\|\theta\|^2 \\
\text{subject to} \quad & y_i \theta^\top x_i \geq 1, \quad \forall i
\end{aligned} \tag{3}$$

Note

There are two reasons to minimize $||\theta||$. First, it is always good to choose the "simplest" solution, and we have already seen in linear regression with the ridge regularizer that having a small norm is a way of enforcing simplicity. Second, there is a strong relation between $||\theta||$ and $\gamma(\theta)$, as we explain in the following Proposition.

Proposition 2.1. *Consider any $\hat{\theta}$ that solves the optimization problem (3), then the margin of $H_{\hat{\theta}}$ is optimal, i.e., $\gamma(\hat{\theta}) = \gamma^*$.*

Proof. Consider any θ that does not make a margin error on the points. We have the following:

$$\gamma(\theta) = \min_i \frac{|x_i^T \theta|}{||\theta||} = \min_i \frac{y_i x_i^T \theta}{||\theta||} \geq \frac{1}{||\theta||}. \tag{4}$$

Let's now look at the hyperplane with the largest margin γ^*. We can select the corresponding vector θ^* such that $\min_i y_i x_i^T \theta^* = 1$ (indeed, it is always possible to rescale θ^* without changing the corresponding hyperplane). We then have the following:

$$\gamma^* = \gamma(\theta^*) = \min_i \frac{|x_i^T \theta^*|}{||\theta^*||} = \min_i \frac{y_i x_i^T \theta^*}{||\theta^*||} = \frac{1}{||\theta||}. \tag{5}$$

We have all the ingredients to prove the statement:

$$\begin{aligned}
\gamma(\hat{\theta}) &\geq \frac{1}{||\hat{\theta}||} && \text{(By Equation (4))} \\
&\geq \frac{1}{||\theta^*||} && (\hat{\theta} \text{ has smallest norm because it solves Problem (3))} \\
&= \gamma^*. && \text{(Because of Equation (5))}
\end{aligned}$$

This concludes the proof. □

What can we say when the underlying dataset is not linearly separable? We introduce some slackness. For any point i, we associate a slackness variable $\xi_i \geq 0$ and we use them to margin error: $x_i^T \theta y_i \geq 1 - \xi_i$. Problem (3) then becomes:

$$\begin{aligned}
\min_{\boldsymbol{\theta}, \boldsymbol{\xi}} \quad & \frac{1}{2}\|\boldsymbol{\theta}\|^2 + C \sum_{i=1}^n \xi_i \\
\text{subject to} \quad & y_i \boldsymbol{\theta}^\top \mathbf{x}_i \geq 1 - \xi_i, \quad \forall i \\
& \xi_i \geq 0, \quad \forall i
\end{aligned}$$

Note, C is a hyperparameter that tunes the relative importance of the norm term and of the margin error. We can further simplify the optimization problem, by noting that, for any fixed vector θ, the best choice of the slackness parameters (i.e., the choice that minimizes the objective while respecting the constraints) is choosing the minimal ξ_i that still respect the constraint, namely, $\xi_i = (1 - y_i \theta^T x_i)_+$, where $(\cdot)_+$ denotes the positive part, and $(a)_+ = \max\{a, 0\}$. Therefore, we can rewrite the equivalent (unconstrained) optimization problem that corresponds soft-margin linear classification:

$$\min_{\boldsymbol{\theta}} \quad \frac{1}{2}\|\boldsymbol{\theta}\|^2 + \frac{C}{n} \sum_{i=1}^n (1 - y_i \boldsymbol{\theta}^\top \mathbf{x}_i)_+ \tag{6}$$

The function $h(z) = (1-z)_+$ is called the hinge function, and is convex. Therefore, the objective function of Problem (6) is convex, and we can safely use gradient or stochastic gradient descent. There is a small caveat: there are some points where the gradient may not be well defined (because the hinge function in 1 does not admit a derivative). In those points, we can choose any vector that still points in the "right direction". This is formalized in the following definition.

Definition 2.2. *Let $F : \mathbb{R}^d \to \mathbb{R}$ be a convex function, and $x \in \mathbb{R}^d$. We say that a vector $g \in \mathbb{R}^d$ is a sub-gradient in x, if the following inequality holds for any $y \in \mathbb{R}^d$:*

$$f(y) \geq f(x) + g^T(y - x)$$

In our case, the sub-gradient of the loss function in a generic θ is $g(\theta)$ defined as follows:

$$g(\theta) = \theta - \frac{C}{n} \sum_i y_i x_i \mathbb{1}_{\{y_i \theta^T x_i \geq 1\}}$$

At this point, it is instructive to look at the pseudocode for the stochastic (sub-)gradient version:

Stochastic Gradient Descent for Support Vector Machine

Input: Training data $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$
Hyperparameters: Max. number of iterations N, learning rates η_t, regularizer parameter C
Initialize $\theta_0 \leftarrow \mathbf{0} \in \mathbb{R}^d$
for $t = 1, 2, \ldots, N$ **do**
  Let i_t be an index drawn uniformly at random in $1, \ldots, n$
  if $y_{i_t} \cdot x_{i_t}^T \theta < 1$ **then** ▷ Margin Error
    $\theta_{t+1} \leftarrow \theta_t - \eta_t(\theta_t - C y_{i_t} x_{i_t})$ ▷ θ_t is adjusted according to point (x_{i_t}, y_{i_t})
  else
    $\theta_{t+1} \leftarrow \theta_t(1 - \eta_t)$ ▷ θ_t is only scaled
return θ

A natural choice for the learning rate is $\eta_t = \frac{1}{Ct}$.

Note

The stochastic sub-gradient descent algorithm for SVM is extremely similar to the Perceptron Algorithm, even though they are designed for slightly different problems. Moreover, this model is called Support Vector Machine (SVM), because the decision boundary only depends on the points considered, where margin errors have been made.

2.1 Kernel Trick

Now, imagine that we want to go beyond linear separators. A possibility may be to map the points to a feature space via a feature map $\psi : \mathbb{R}^d \to \mathbb{R}^{d'}$. We do not want to find an algorithm that depends on the feature dimension d', which may be infinite or very large. Since we want a definition that also covers infinitely dimensional vector space, we state the formal definition of scalar products, that generalizes the product $a^T b$ for vectors in finite dimensions.

Definition 2.3. *Given a vector space $\mathbb{V}$, a scalar product $\langle \cdot, \cdot \rangle : \mathbb{V}^2 \to \mathbb{R}$ satisfies the following properties:*

- ***Linearity**: for any vectors $v, w, z \in \mathbb{V}$ and scalar $\alpha \in \mathbb{R}$, it holds*

$$\langle \alpha v, z \rangle = \alpha \langle v, z \rangle \text{ and } \langle v + w, z \rangle = \langle v, z \rangle + \langle w, z \rangle.$$

- ***Simmetry**: for any vectors $v, w \in \mathbb{V}$ it holds $\langle v, w \rangle = \langle w, v \rangle$.*

- ***Positivity**: for any vector $v \in \mathbb{V}$, $\langle v, v \rangle \geq 0$, and the equality is only reached for $v = 0$.*

It is a simple exercise to see that the standard scalar product in the Euclidean space is indeed a scalar product according to this formal and general definition. The last ingredient we need is the notion of Hilbert space: essentially, an Hilbert Space is a vector space equipped with a scalar product.*

We thus want to exploit maps to feature spaces that are Hilbert Spaces. There are two crucial observations underlying the so-called Kernel Trick, that allow for a feasible running time:

- The vector θ_t maintained by SVM is always a linear combination of vectors of the form $\psi(x_1), \ldots, \psi(x_n)$. Therefore all the optimization procedure lives in this n-dimensional subspace of the target Hilbert Space. Therefore we only need to maintain the n coordinates $\alpha_1^t, \ldots, \alpha_n^t$ in memory, that identify $\theta_t = \sum_{i=1}^n \alpha_i^t \psi(x_i)$
- We actually do not need to know exactly θ_t, but only to be able to compute scalar products of the type $\langle x_i, \theta_t \rangle$. By the properties of the scalar product, we can rewrite it as follows:

$$\langle x_i, \theta_t \rangle = \langle x_i, \sum_{j=1}^n \alpha_j^t \psi(v_j) \rangle = \sum_{j=1}^n \alpha_j^t \langle \psi(x_i), \psi(x_j) \rangle.$$

Therefore, at the end of the day, we only need to maintain in memory the n coordinates of θ, and the so-called kernel matrix K whose generic element is given by $K(x_i, x_j) = \langle \psi(x_i), \psi(x_j) \rangle$ (so we assume that we know how to compute the feature function on the points, and that we know how to compute the scalar products!).

Example 2.4 (Polynomial Kernel)**.** *Imagine that the dimension of the input space is $d = 1$, and we consider the feature maps $\psi_i(x) = x^i$, for $i = 0, \ldots, D$. The target space is still an Euclidian vector space of dimension $D + 1$, and the scalar product becomes $\langle \psi(x), \psi(z) \rangle = \sum_{i=0}^D x^i y^i$. This can be generalized to multiple dimensions, and arbitrary degree. Note, by the kernel trick you only need to maintain an $n \times n$ Kernel Matrix, and the n coordinates of θ_t.*

Example 2.5 (Gaussian Kernel)**.** *The dimension of the input space is d, we can map the points to an infinite dimensional vector space whose scalar product is computed as follows: $\langle \psi(x), \psi(z) \rangle = e^{-||x-z||^2/2}$. This type of Kernel is called Gaussian or Radial Basis Function (RBF) Kernel.*

3 Pointers

The discussion on the Perceptron is mainly based on Hardt and Recht [2022], while that on Support Vector Machine on Shalev-Shwartz and Ben-David [2014] and Géron [2019].

References

Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.* O'Reilly Media, Inc, second edition edition, 2019.

Moritz Hardt and Benjamin Recht. *Patterns, predictions, and actions: Foundations of machine learning.* Princeton University Press, 2022. URL `https://mlstory.org/pdf/patterns.pdf`.

*Formally, a vector space with a scalar product is an Hilbert Space if it is complete with respect to the metric induced by the scalar product, but we are not being that picky here.

Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press, 2014. URL https://dn721802.ca.archive.org/0/items/understanding-machine-learning-theory-algorithms_202404/understanding-machine-learning-theory-algorithms.pdf.